

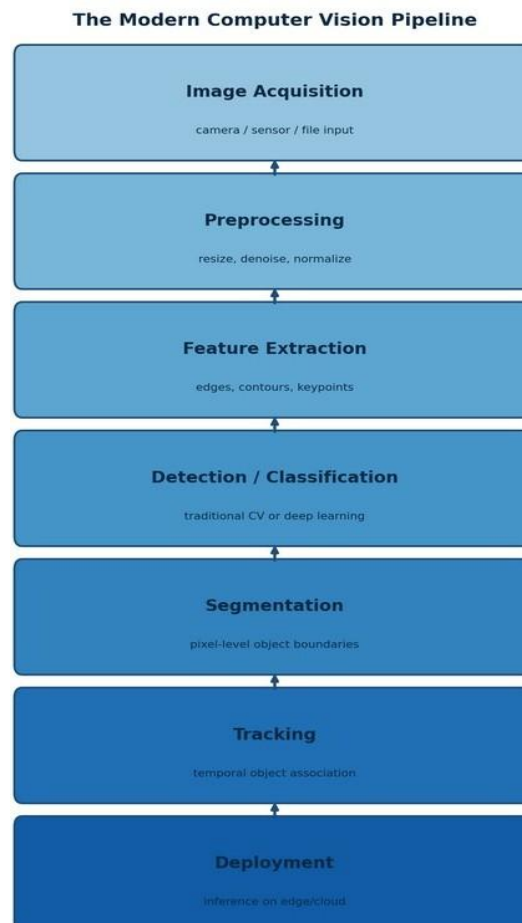
The Modern Computer Vision Pipeline

Technical Research Report — AI Summer Internship 2026

1. Introduction

Computer Vision (CV) is the field of engineering systems that extract meaning from images and video. A production CV system is rarely a single algorithm; it is a pipeline of discrete stages, each converting data into a more structured representation than the last, until the output is something an application can act on. This report walks through that pipeline stage by stage: image acquisition, processing, detection, classification, segmentation, tracking, and deployment.

2. Pipeline Overview



3. Image Acquisition

This stage captures raw visual data from a sensor: a webcam frame, a stored image file, an IP camera stream, or a video feed. The output is a raw pixel array (height x width x channels). Engineering concerns here include resolution, frame rate, color space (RGB vs BGR, as OpenCV defaults to BGR), and bit depth. A poor acquisition setup (low light, motion blur, wrong exposure) caps the quality of every downstream stage — no amount of processing later fully compensates for a bad capture.

4. Image Processing (Preprocessing)

Raw pixel data is rarely fed directly into a detector. Preprocessing standardizes and cleans the input:

- Resizing / normalization — standardizes input dimensions and pixel value ranges for consistent downstream behavior.
- Grayscale conversion — reduces 3-channel color data to 1 channel when color is not discriminative, cutting compute cost.
- Noise removal (Gaussian/Median blur) — suppresses sensor noise that would otherwise be misread as edges.
- Thresholding — converts grayscale intensity into binary foreground/background masks (global or adaptive).
- Edge detection (Canny) — highlights sharp intensity gradients that typically correspond to object boundaries.
- Morphological operations (erosion, dilation, opening, closing) — clean up binary masks by removing small noise blobs or closing small gaps.

A frequently tested engineering fact: edge detectors are highly sensitive to noise, so a blur pass is applied before Canny edge detection — without it, noise pixels get misclassified as false edges.

5. Feature Extraction

Once the image is cleaned, the pipeline extracts structured signals: contours (continuous boundary curves), keypoints (corners, blobs — via detectors such as ORB, SIFT, or Harris corners), and histograms (color/intensity distributions). Traditional CV relies on hand-engineered features like these; deep learning approaches instead learn feature representations directly from data through convolutional layers, which is the central difference between the two paradigms.

6. Object Detection

Detection answers 'where are the objects, and what are they' — producing bounding boxes with class labels and confidence scores. Classical approaches (Haar cascades, HOG + SVM) rely on hand-crafted features and sliding windows. Modern deep-learning detectors (YOLO, SSD, Faster R-CNN) learn features end-to-end and dominate production use today due to higher accuracy and better generalization across lighting, scale, and occlusion.

7. Classification

Classification assigns a single label to an entire image or a cropped region ('this is a cat'), as distinct from detection, which both localizes and labels multiple objects in one frame. Classification is typically the simpler sub-problem and is often used as a downstream step after a detector has already isolated a region of interest.

8. Segmentation

Segmentation extends detection to pixel-level precision. Semantic segmentation labels every pixel with a class (all 'road' pixels, all 'sky' pixels) without distinguishing instances. Instance segmentation goes further, separating individual object instances (car #1 vs car #2) with pixel-accurate masks, as used in architectures like Mask R-CNN. Segmentation is essential wherever exact object shape matters — medical imaging tumor boundaries, autonomous vehicle drivable-area mapping, and background removal tools.

9. Tracking

Tracking associates detected objects across consecutive video frames over time, assigning persistent IDs so the same object is recognized frame to frame rather than re-detected from scratch. Classical trackers (Kalman Filter, Optical Flow, CSRT) predict short-term motion; modern deep-learning trackers (DeepSORT, ByteTrack) combine a detector with a learned appearance embedding for robust re-identification through occlusion. Tracking is what turns a frame-by-frame detector into a system that understands motion, count, and trajectory.

10. Deployment

The final stage moves a trained/tuned pipeline from a notebook into a running system: exporting models to inference-optimized formats (ONNX, TensorRT, TFLite), serving via an API or embedding directly in an edge device, and monitoring for drift and latency in production. Deployment constraints — inference speed, memory footprint, and hardware (GPU vs CPU vs edge NPU) — often force real trade-offs against the accuracy achieved during training.

11. Real-World Applications

- Autonomous vehicles — lane/object detection, segmentation of drivable area, multi-object tracking of pedestrians and vehicles.
- Medical imaging — tumor/organ segmentation in MRI/CT scans for diagnosis support.
- Retail — shelf-inventory detection, customer footfall tracking, self-checkout item recognition.
- Security & surveillance — intrusion detection, face recognition, crowd counting.
- Industrial inspection — defect detection on manufacturing lines via classification and segmentation.
- Agriculture — crop health classification and yield estimation from drone/satellite imagery.

12. Conclusion

The modern CV pipeline is a chain of increasingly abstract representations: pixels to features, features to detections, detections to tracked, segmented, and finally deployed intelligence. Understanding each stage — and why it exists relative to the one before it — is what separates someone who can call `cv2.Canny()` from an engineer who can design and debug a full vision system.

References

- OpenCV Documentation — <https://docs.opencv.org/>
- Szeliski, R. — Computer Vision: Algorithms and Applications, Springer.
- Redmon, J. et al. — You Only Look Once: Unified, Real-Time Object Detection (YOLO paper), CVPR 2016. He, K. et al. — Mask R-CNN, ICCV 2017.
- Wojke, N. et al. — Simple Online and Realtime Tracking with a Deep Association Metric (DeepSORT), 2017.

